Appendix B — Reflective Summary on AI Use

I used Claude (Anthropic) throughout this project as a design sounding board and code reviewer, not as a code generator. For almost every class, I wrote the first version myself, then shared it for review, where the AI's job was to catch mistakes and question decisions, similar to peer review work, not to produce working code for me to paste in. The handful of cases where I did have it generate code directly were ones where a pattern had already been established across earlier classes or methods, and what remained was applying that same semi-repeated structure to the rest (the file handlers' reconstruction constructors once the pattern was set on the first one, or the menu classes once the shared base class and navigation pattern were in place). AI never had direct access to my codebase. Every change went through me typing it into NetBeans myself, and even generated classes were only pasted in after close reading and reviewing.

The most useful pattern across the project was being pushed to defend my own design decisions with real detail rather than abstract reasoning. Early on, I proposed a Vehicle class hierarchy with a subclass per category (Compact, Intermediate, Standard, and so on). The AI's first response was cautious but open to it. It was only once I gave a concrete example, that the same category, Intermediate, actually contains vehicles with completely different GPS and fuel combinations in real fleets, that it became clear the variation was at the level of an individual vehicle, not the category. That single example was enough to collapse what would have been an unnecessary five-class hierarchy into one enum. The same thing happened with Staff: I had planned subclasses per role, but once I explained the only real difference was permissions, not data, the design correctly moved to a StaffRole enum instead. In both cases the AI's first suggestion was reasonable given what I'd told it, and it changed once I gave it the domain knowledge it didn't have. That discussion dynamic was exactly what I needed from an AI tool. It was a way to pressure-test my thinking and arrive at a genuinely concrete design.

The other place AI usage was genuinely valuable was in catching a specific, recurring class of bug I would likely have missed reading my own code back: constructor parameters that don't quite match the field they're meant to assign to, silently self-assigning a field to itself instead. This happened several separate times across the project, in different classes, and each time it was caught immediately during review because it's an easy pattern to check for mechanically, even though it's easy to miss visually. A second recurring bug type was comparing an enum value against a plain string instead of the enum constant itself. This compiles fine and never throws an error, it just silently matches nothing, which is a genuinely hard bug to notice without someone specifically looking for it.

Beyond design and review, I used Claude Code separately to run smoke tests against the finished service and UI layers, treating it as an independent tester rather than part of the build process. This surfaced two real bugs that had nothing to do with typos or syntax: a vehicle's rental rate had no path to ever become non-zero, since nothing in the service layer ever set it, and locations were never actually being written to disk, meaning the whole application would silently lose location data the moment it restarted. Both were the kind of structural gap that's easy to miss when you're the one who wrote the code, since I already knew what I meant a given method to do.

AI also made it easier to organize and check the AI usage log itself, and to write this reflective summary. English is my second language, and although I am fluent, using AI here helped me express things more clearly and accurately.

Overall, AI use on this project was concentrated on architecture discussions and code review rather than code generation, and the value came mostly from being asked to justify decisions and having a second pass over code looking specifically for the kinds of mistakes that don't show up as compiler errors.